

CS426

Mobile Device Application Development

Final Project Report

Mobile Image Retrieval

Design, Implementation, and Evaluation

An On-Device Photo Retrieval Application

Team Member	Student ID
Vong Chi Van	24125049
Le Dat Phuc An	24125052
Pham Nguyen Minh Quan	24125041
Tran Canh Anh Tuan	24125048

Application version: 1.0

Target platform: Android 10 (API 29) and above

Abstract

The growth of personal photo libraries creates a retrieval problem when users remember an image’s content but cannot recall its filename, capture date, or album. This project addresses that problem through Mobile Image Retrieval, a native Android application that supports semantic text queries, reference-image queries, explicitly enrolled people, and optical character recognition (OCR). The intended users include individuals managing mixed personal collections and users retrieving receipts, messages, and screenshots.

The application combines MobileCLIP2-S0 for visual retrieval, SCRFD and MobileFaceNet for face detection and recognition, and bundled ML Kit text recognition for literal text search. Its architecture separates Jetpack Compose interfaces, ViewModel state management, repository coordination, and persistent storage. Room maintains a local index, while MediaStore supplies access to device photos and WorkManager schedules incremental indexing. Inference is performed on the device; the application manifest excludes network permissions.

The implemented system provides eight connected feature screens and supports retrieval, filtering, photo viewing, sharing, saving copies, and deletion through Android media services. The principal limitations concern model size, initialization cost, recognition errors, and the current restriction to image media. **Keywords:** Android; on-device inference; image retrieval; semantic search; face recognition; OCR; local persistence.

Report Organization

Sections 1–5 establish the application context, work allocation, requirements, and screens. Sections 6–13 describe usability, architecture, implementation, and technology. Sections 14–15 document setup and installation. Sections 16–18 present validation, self-assessment, and limitations.

Contents

1	Introduction and Project Objectives	1
2	Target Users and Use Cases	1
3	Work Allocation and Collaboration	2
4	Requirements Analysis and Scope	3
5	Application Screens and Navigation	4
6	User Interface and Usability	5
7	Application Architecture	8
8	Data Persistence and Schema Migration	9
9	Semantic and Multimodal Retrieval	10
10	Person Enrollment and Face Matching	11
11	OCR and Text Retrieval	12
12	Incremental Indexing and Reliability	12
13	Technology Stack and Configuration	13
14	Development Environment and Build Procedure	14
15	Installation and Operating Procedures	15
16	Testing Methodology and Validation Results	16
17	Self-Assessment Against the Course Rubric	18
18	Limitations and Future Development	18

1 Introduction and Project Objectives

1.1 Problem Statement

Personal photo libraries commonly include camera photographs, screenshots, receipts, message captures, and downloaded images. Filename-based and chronological browsing provide limited support when a user recalls visual content or a textual phrase but cannot identify the corresponding file or capture date. Manual categorization also introduces a continuing organizational burden.

The topic of this project is **private, on-device image retrieval**: making an Android photo library searchable through visual concepts, examples, known people, and recognized text. The application operates on accessible device photos and produces results that the user can open and use immediately. It requires no account, application server, or cloud photo upload.

1.2 Objectives and Acceptance Criteria

Table 1. Project Objectives and Acceptance Criteria

Objective	Observable success condition
Visual content retrieval	A text prompt, reference image, or combined query returns ranked device photos.
Enrolled-person retrieval	A saved handle filters results by face similarity; multiple handles require distinct matching faces.
Text-based image retrieval	OCR search retrieves photos containing specified words or numbers.
Retrieval-to-action workflow	Retrieved photos open in a viewer supporting zoom, sharing, copying, deletion, and text reading.
Persistent application state	Indexed data, saved people, and text-search history are retained across application restarts.

1.3 Scope and Contribution

The implemented product focuses on accessible **images**. The database model includes a video type, but the current application does not request video permission or provide video indexing. Albums support browsing device collections; creating or editing albums is outside the implemented scope.

The project integrates three retrieval mechanisms with different purposes. CLIP represents visual meaning; face embeddings represent enrolled identities; OCR indexes literal text. Keeping these representations separate avoids treating a person’s identity as a scene description or treating recognized words as visual relevance.

The principal engineering contribution is the integration of pretrained retrieval models with Android permission management, lifecycle handling, background execution, persistent storage, and media operations. Evaluation addresses implemented functionality and documented reliability scenarios. Recognition quality and performance across representative devices remain subjects for further empirical assessment.

2 Target Users and Use Cases

2.1 Intended Audience

The primary target group comprises Android users managing heterogeneous personal image collections who require content-based retrieval without comprehensive manual tagging. English visual descriptions remain the most suitable input for Normal query.

The following profiles are capability-based design scenarios. Empirical validation through interviews or usability studies remains outside the available evaluation evidence.

Table 2. Target User Profiles and Supported Workflows

User profile	Need	Supported workflow
Student	Recover a saved receipt or message screenshot without remembering its date.	OCR search → enter “invoice” → open result → Read text.
Personal photo owner	Find a scene, object, or visually similar image among many photos.	Normal query → enter a description or choose a reference image → browse ranked results.
Family or travel user	Find an enrolled person within a meaningful setting.	People → save one clear face → search “@alex at the beach.”
User with limited connectivity	Browse and search accessible photos without uploading the library.	Install an APK containing the models → grant access → index → search offline.

2.2 Representative Use Cases

1. **Recover a document image.** The user selects OCR search and enters words actually visible in a screenshot. The application returns matching photos and preserves the original recognized text in the reader. The user copies the required passage.
2. **Find a remembered scene.** The user enters “sunset at the beach” in Normal query. After indexing, the application ranks photos by visual similarity. Date filters narrow the candidates, and the viewer exposes share and save-copy actions.
3. **Find known people together.** The user enrolls two people separately and searches their handles together. Each person must match a distinct detected face. Adding a visual description further ranks the photos that pass the identity filter.
4. **Reuse a found image.** The user opens a result, selects Save a copy, and follows View to the new image in Recently Added. The original photo remains in place.

3 Work Allocation and Collaboration

3.1 Team Composition

The final project for CS426: Mobile Device Application Development is submitted by the following four-member team:

Team Member	Student ID
Vong Chi Van	24125049
Le Dat Phuc An	24125052
Pham Nguyen Minh Quan	24125041
Tran Canh Anh Tuan	24125048

3.2 Attribution Status

The table below proposes a randomized, balanced allocation: each member receives two primary work packages, and testing is shared equally by all four members. Each member therefore has a 25% share of the planned allocation, counting work packages equally. This allocation is a planning assignment; it does not establish historical contribution or measured effort.

Table 3. Work Allocation and Deliverable Status

Work package	Member	Deliverable and completion evidence
Requirements and product scope	Tran Canh Anh Tuan	Requirement mapping, topic, use cases, and demonstration sequence in COURSE_REQUIREMENTS.md and README. Documentation exists.
Compose UI and navigation	Pham Nguyen Minh Quan	Eight feature screens, adaptive grids, viewer interactions, permission states, saved-state handling. Implemented in MainActivity and ui/.
Data and Android integration	Tran Canh Anh Tuan	Room schema/migrations, MediaStore browsing, history, copying, sharing/deletion integration. Implemented in data/ and MainActivity.
Visual retrieval	Vong Chi Van	Export contract, CLIP preprocessing/tokenization, embeddings, exact top-K, image/text combination. Implemented in ai/ and export tooling.
Face enrollment and search	Vong Chi Van	SCRFD/MobileFaceNet integration, alignment, references, distinct-person matching, versioned caches. Implemented in ai/ and repositories.
OCR and text retrieval	Le Dat Phuc An	Bundled recognition, FTS4 search, text reader. Implemented in OCR/data/UI components.
Background indexing	Le Dat Phuc An	Incremental worker, scheduling, and consistency protection. Implemented in worker/ and repositories.
Report and presentation	Pham Nguyen Minh Quan	Report source and demonstration protocol prepared. Final artifact review and presentation delivery remain pending.
Testing and validation	All four members (equal share)	Unit and instrumentation tests for retrieval, Room persistence, OCR, indexing, and media operations; build/lint checks and device regression testing. Existing tests and audit results provide the baseline; record new outcomes and defects.

3.3 Operating Assumptions

The operating assumptions are an Android 10+ device, sufficient storage for model assets and the local index, and user-authorized access to the relevant photos. Approximately 5,000–30,000 images as the intended library range; sustained performance at that scale requires device-level profiling. Incremental indexing permits initial use with a smaller collection.

Face search is an aid to personal photo retrieval. Matching can miss a person or confuse similar faces, and the displayed visual match percentage is a relevance indicator rather than a probability. No user study or calibrated recognition benchmark is supplied in the repository.

4 Requirements Analysis and Scope

4.1 Required Application Capabilities

The course specification requires at least three to four functionally connected screens, persistent local storage, integration with a device capability or external data source, an approved Android development stack, and a native-installable application. The implementation mapping below identifies the corresponding components and verification procedures.

Table 4. Traceability of Course Requirements

Requirement	Implemented evidence	Verification route
Connected screens	Eight feature screens connect search, results, filters, people, albums, album contents, viewing, and text reading.	Follow Section 5; inspect MainActivity.
Persistent local data	Room schema 4 stores media metadata and embeddings, faces, OCR, people, history, and indexing state.	Reopen the same installation; inspect database entities and migrations.
Device or external integration	MediaStore reads actual device photos. The system photo picker, share sheet, save-copy operation, and deletion flow integrate with Android.	Browse albums and perform viewer actions on device photos.
Permitted stack	Kotlin, Android SDK, Jetpack Compose, ViewModel, Room, and WorkManager.	Inspect Gradle configuration and source packages.
Native Android product	Gradle assembles an installable APK; minSdk is 29. Local inference models are packaged as assets or bundled dependencies.	Build and install using Sections 13–14.

Device integration satisfies the supplied alternative to an external web API. A REST backend is not necessary for the application’s purpose, and no server deployment is part of the setup.

4.2 Non-Functional Requirements

- **Responsiveness:** searches use coroutines; expensive inference is outside UI rendering; WorkManager controls incremental indexing.
- **Persistence and integrity:** completed stages are committed per photo, migrations retain earlier data, and revision checks reject stale writes against a newer photo version.
- **Usability:** the UI distinguishes missing access, empty libraries, loading, queued work, errors, and partial results. Grids adapt to width, and viewer actions remain reachable in landscape.
- **Privacy:** inference and indexing remain local; the manifest removes network permissions and disables backup. Sharing is a separate, user-initiated Android action.

4.3 Assessment Scope

The assessment scope comprises application implementation, technical quality, interface design, originality, documentation, presentation, and collaboration.

5 Application Screens and Navigation

The application uses a single Activity with Navigation Compose. Search, People, and Albums form the top-level destinations; Results, Filters, Photo Viewer, and Text Reader complete the principal retrieval and browsing workflows. Loading and permission views are treated as interface states rather than independent feature screens.

People → Search → Results ↔ Filters
Results → Photo viewer → Text reader
Albums → Album contents → Photo viewer

Figure 1. Feature-screen navigation. Arrows indicate the principal user workflows; back navigation is omitted for clarity.

Table 5. Application Screens and Responsibilities

Screen	Purpose and connection
Search	Select Normal query or OCR search, enter text, choose a reference image, and access history.
Results	Display a grid of real matches; open a photo or adjust the active search.
Filters	Apply predefined/custom dates and sorting; update the next query or rerun Results.
People	Add, update, and remove face references; use saved handles in a query.
Albums	Browse actual MediaStore buckets and system collections independently of AI indexing.
Album contents	Open photos belonging to the selected collection.
Photo viewer	Inspect and zoom; find similar photos, share, save a copy, delete, or read text.
Text reader	View, select, and copy the original recognized text.

The viewer has routes for results, the library, and album contexts. Save a copy produces a snackbar with a View action that opens the new media item through Recently Added. “Find similar photos” makes the current photo the next query’s image reference.

5.1 Search Screen and Library Access

The Search destination brings query entry, retrieval-mode selection, indexing status, and recent searches into one screen. The first screenshot shows Normal query selected, with controls for an image query, saved people, and a date filter. Suggested queries help users begin a search, while the indexed-photo count and Refresh action communicate library readiness.

The second screenshot shows the lower portion of the same destination. Recent searches display query text, relative times, and thumbnails where available. Below them, the Your library grid provides direct access to photos. The persistent bottom navigation connects this workflow to Albums and People.

5.2 Album Browsing and Person Enrollment

Albums provides a collection-based route to photos. Its screenshot lists All Photos, Recently Added, and Camera, each with a preview and photo count. Selecting a collection leads to its contents, from which the user can open the photo viewer. The refresh control updates the displayed library collections.

People manages the identities used in person-filtered queries. The screenshot shows an enrolled entry named alex, its @alex handle, a face thumbnail, and controls to change the face photo or remove the entry. Add person starts enrollment, and the face-indexing count indicates progress. After enrollment, the user can return to Search and combine a saved handle with a scene description. The displayed counts describe the captured library state.

6 User Interface and Usability

6.1 Query Modes and User Feedback

The query interface distinguishes two retrieval modes. **Normal query** accepts visual text, an image, or both. **OCR search** accepts words and numbers from image text; it rejects image-query input and does not invoke the CLIP text encoder. Both modes can apply saved-person filters when the required face data is available.

Separate progress indicators represent OCR and visual/face indexing. Completed items become searchable before the full library has been processed. WorkManager state distinguishes queued tasks from

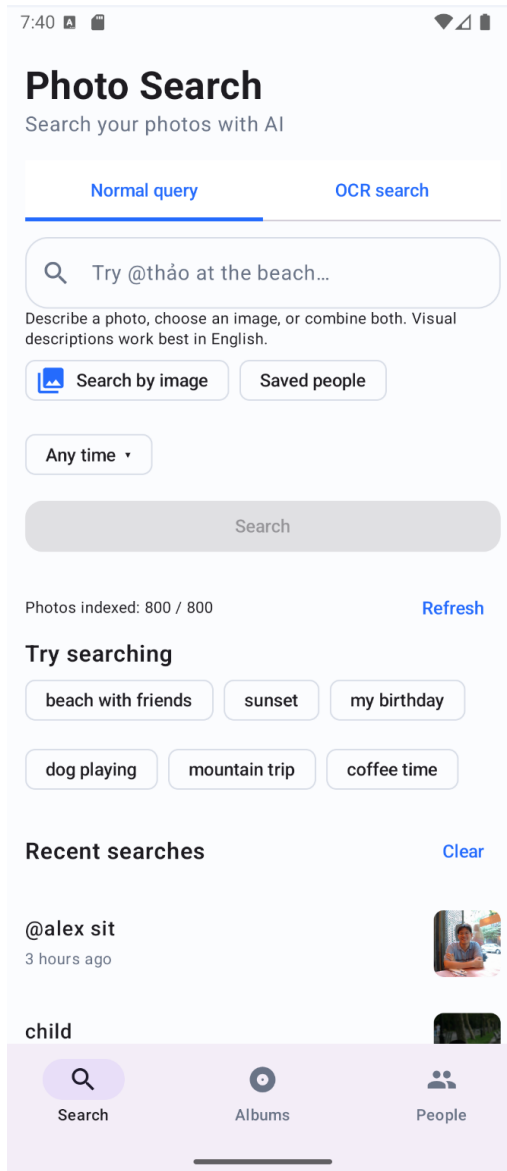


Figure 2. Search entry: query modes, image input, date filtering, indexing progress, and suggested queries.

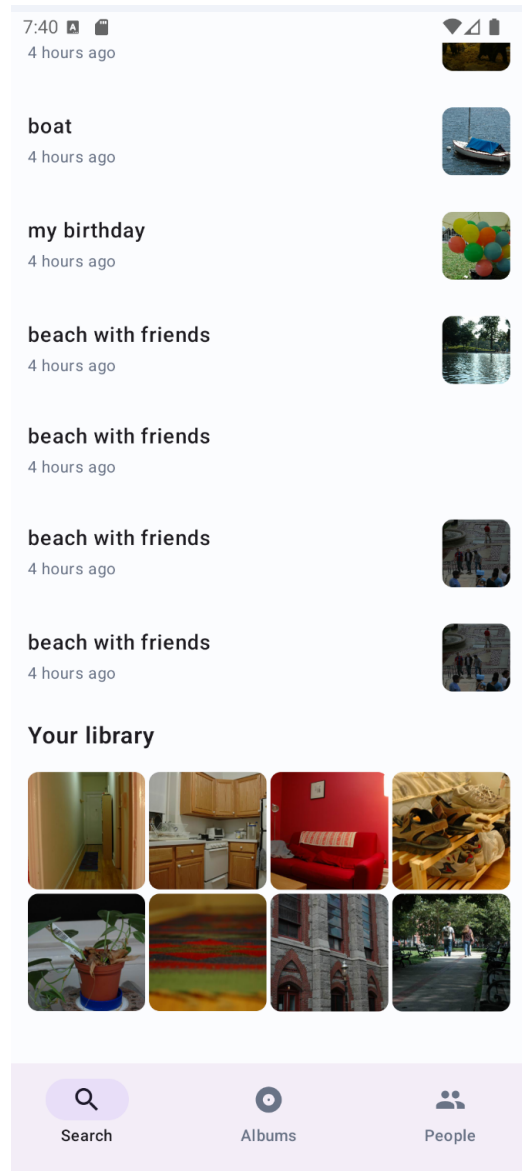


Figure 3. Search screen scrolled to recent queries and the device-photo library grid.

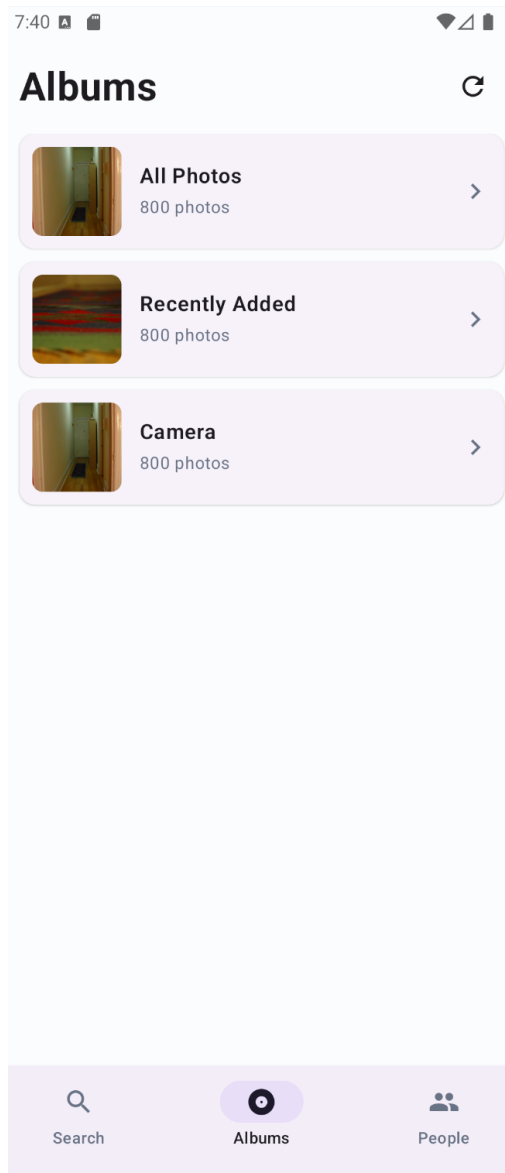


Figure 4. Albums: collection previews and photo counts provide an entry point for browsing device images.

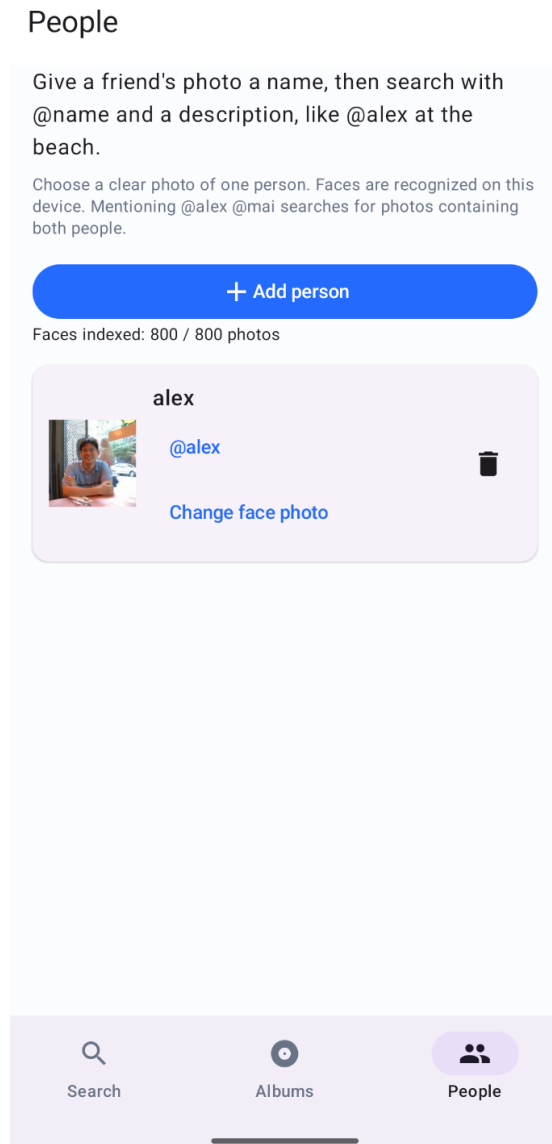


Figure 5. People: explicit enrollment and saved-face management support queries using named handles.

active execution, including delays caused by battery constraints. Refresh and Search again enable the user to retry pending work and retrieve newly indexed items.

Table 6. Interface States and Recovery Mechanisms

Situation	Interface response	User benefit
No photo access	Permission guidance and recovery through app settings.	Explains why photos are unavailable.
Selected-photo access	Partial-access state with controls to add access.	Clarifies the searchable library’s scope.
Empty library or no matches	A distinct empty state.	Separates missing content from a read failure.
Image loading fails	Retry action.	Supports temporary URI or decoding problems.
Missing/inconsistent model	Visible model error.	Makes asset problems actionable.
Unknown or outdated person	Prompt to add a person or update their face photo.	Prevents silent use of invalid references.
Copy in progress or failure	Disable repeat save; show success or error feedback.	Avoids duplicate operations and explains outcomes.

6.2 Adaptive Layout and Interaction

Photo grids choose their column count from available width. The viewer supports pinch/pan, double-tap zoom, and resetting zoom. In landscape it places the image beside scrollable actions. The application follows system light/dark appearance, and saved-person removal requires confirmation.

6.3 Lifecycle Continuity

Search drafts and filter edits use saved state. Navigation requests remain in ViewModel state until handled, including a search that finishes during rotation. After process death, transient search/result destinations return to the restored draft; persisted people, history, and indexes remain in Room. An interrupted foreground search is restarted from the restored query rather than resumed at an intermediate execution point.

7 Application Architecture

The application adopts a **Model–View–ViewModel (MVVM) architecture with repository-based data access**. Compose renders state and sends actions to SearchViewModel. Repositories coordinate persistence, Android media access, and AI services. AppContainer constructs these dependencies once for the application; there is no dependency-injection framework or remote service layer.

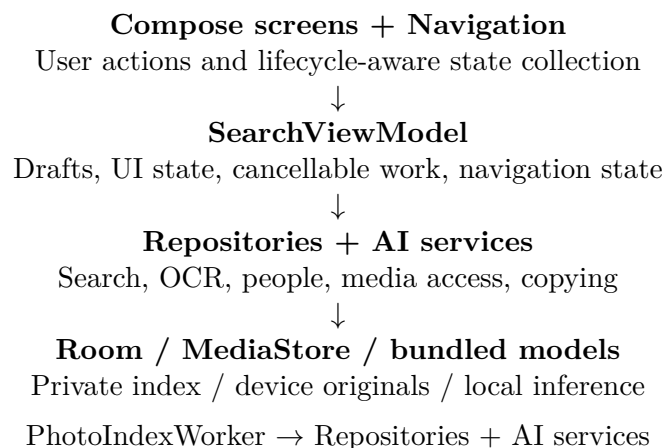


Figure 6. Application layers and dependency direction. Observable state returns to the interface; background indexing uses the same repositories through AppContainer.

Table 7. Software Components and Responsibilities

Package / component	Responsibility
ui/ and MainActivity	Compose screens, navigation, lifecycle-aware collection, user feedback, ViewModel orchestration.
domain/model/ data/repository/	Media items, filters, results, albums, indexing states, and typed UI errors. Search/history coordination, image references, face indexing, text caching, incremental planning.
data/db/ data/mediastore/	Room entities, DAOs, migrations, transactions, and embedding serialization. Device metadata, thumbnails, library changes, and byte-preserving photo copies.
ai/ and worker/	Model contracts, preprocessing, inference, retrieval, and resumable indexing work.

This separation allows mathematical ranking and incremental planning to be tested without the full UI. SearchCandidateSource abstracts candidate access, while model interfaces allow repository tests to substitute deterministic encoders. UI behavior, Room, and real bundled inference are also exercised through instrumentation tests.

8 Data Persistence and Schema Migration

Room opens `photo-search.db` in application-private storage at schema version 4. MediaStore remains the owner of the original photos. The image index stores content URIs and metadata rather than filesystem paths or copies of full-resolution image bytes. Saved people separately retain a small JPEG reference thumbnail.

Parent / source	Relation	Child / target
media_embeddings	→	photo_text
media_embeddings	→	face_index
media_embeddings	→	face_embeddings
photo_text	→	photo_text_fts

Independent tables: `people`, `search_history`, `indexing_state`.

Figure 7. Database relationships. Photo-dependent records use cascading foreign keys, while `photo_text_fts` is synchronized with `photo_text`.

8.1 Representation and Lifecycle

A normal image vector contains 512 Float32 values serialized by EmbeddingCodec. At four bytes per value, the raw vector payload is **2,048 bytes per photo**: 20.48 MB for 10,000 photos before metadata and database overhead. Face vectors add storage per detected face; OCR and saved thumbnails add variable amounts.

A media row with `embeddingDimension = 0` represents known metadata awaiting visual inference. Normal background indexing processes visual embeddings and faces before OCR; OCR data is stored separately from the visual vector. Normal search excludes these pending rows. Completing the visual vector for the same revision preserves already-written OCR data.

Table 8. Database Migration History

Migration	Preserved or added behavior
1 $\rightarrow$ 2	Add saved people and a unique handle index while retaining existing semantic data and history.
2 $\rightarrow$ 3	Add per-photo face state and per-face vectors; mark existing person-reference model types for explicit re-enrollment.
3 $\rightarrow$ 4	Add original/normalized OCR text, an FTS4 index and synchronization triggers, plus history search-mode storage.

OCR and face writes compare the current parent revision inside a transaction. An obsolete inference result cannot attach to a newer indexed photo revision. Updating a photo invalidates dependent old data; deleting its parent cascades to its OCR and face records. History insertions retain only the latest 100 text searches. Uninstalling or clearing app data removes this private database.

9 Semantic and Multimodal Retrieval

9.1 Query Preparation

MobileCLIP2-S0 maps text and images into a shared 512-dimensional space. Library images are represented by image embeddings; a query may supply a text embedding, a reference-image embedding, or both. All vectors are L2-normalized.

Text uses the exported OpenCLIP byte-BPE tokenizer contract, including a 77-token context and vocabulary/merges assets. Image preprocessing follows the generated configuration: RGB input, shortest-edge bicubic resize, center crop to 256×256 , and conversion to Float32 NCHW values in the declared range. The two graphs must originate from the same checkpoint/export.

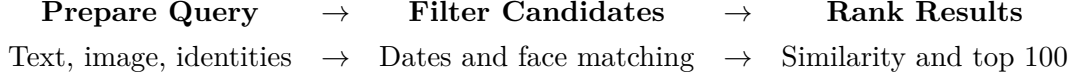


Figure 8. Semantic retrieval pipeline. Person references constrain candidate selection before visual ranking; OCR follows the separate procedure described in Section 10.

9.2 Similarity and Result Selection

For an L2-normalized query vector $\mathbf{q}$ and image vector $\mathbf{x}$, cosine similarity is computed as the dot product. When both modalities are supplied, the normalized text vector $\mathbf{t}$ and reference-image vector $\mathbf{v}$ receive equal weight:

$$\text{score}(\mathbf{q}, \mathbf{x}) = \sum_{i=0}^{511} q_i x_i, \quad (1)$$

$$\mathbf{q}_{\text{combined}} = \frac{0.5\mathbf{t} + 0.5\mathbf{v}}{\|0.5\mathbf{t} + 0.5\mathbf{v}\|_2}. \quad (2)$$

The combined-query expression applies when both text and a selected reference image are present. A single modality uses its own normalized vector. Face embeddings never enter this combination. People-only queries can instead rank by their face-match score.

The engine reads Room candidates in pages of 512 and maintains a bounded min-heap. For N candidates, vector dimension $D = 512$, and result limit $K = 100$, the core scoring and ranking cost is approximately $O(ND + N \log K)$, excluding database access and face matching. Candidate pagination bounds the image-vector working set, and heap selection avoids a complete result sort. OCR match identifiers and index-planning metadata may still scale with library size.

Normal query selects the best 100 relevant photos first. Newest/oldest sorting then reorders those selected matches. Ties use photo dates and media IDs for deterministic results. This differs from OCR chronological search, which selects the correct date-ordered 100 from the complete text-match set.

9.3 Runtime Contract and Interpretation

ONNX sessions are initialized lazily and retained for the process. Inference uses serialization and two intra-op threads to limit CPU contention. The runtime checks configured tensor shapes, types, and tokenizer vectors and reports incompatible assets. A displayed match percentage is a normalized relevance indicator, not model confidence or a measured likelihood that the photo satisfies the query.

10 Person Enrollment and Face Matching

10.1 Enrollment and Local References

People search begins with explicit enrollment. The user opens People, selects one clear photo, and saves a name. Names support 1–40 Unicode letters, numbers, or underscores; spaces become underscores and handles are case-insensitive. Duplicate handles are rejected. A person named “Mai Anh” can be referenced as @mai_anh.

Enrollment requires exactly one detected face, a detection score of at least 0.7, and face dimensions of at least 60 pixels in the bounded decoded image. No-face, group, or unclear selections produce an error. The application stores a reference thumbnail, normalized face vector, and model version. The saved reference is independent of the original picker URI. Change face photo replaces the reference while preserving the name and handle.

10.2 Detection and Representation

Table 9. Face Processing Pipeline

Stage	Implementation
Original decoding	Apply EXIF orientation and bound the longest side to 1280 pixels, retaining the full frame.
Detection	SCRFD <code>det_500m.onnx</code> ; 640×640 RGB input, detection cutoff 0.5, non-maximum suppression IoU 0.4.
Alignment	Use five detected landmarks to align the face to the 112×112 ArcFace template.
Recognition	MobileFaceNet <code>w600k_mbf.onnx</code> ; produce a 512-D L2-normalized face vector.
Caching	Store up to 64 face vectors sequentially per photo plus completion/model-version state, including no-face outcomes.

Face vectors occupy their own model space. They are not compared to CLIP image vectors even though both happen to have 512 dimensions. Old whole-photo person references are marked for Update face rather than silently treated as face-recognition embeddings.

10.3 Multiple-person Semantics

For @alex @mai at the beach, the parser resolves both saved references and removes the mentions from the scene text. Each identity must match a **distinct** face with cosine similarity at least 0.45. Only photos satisfying that condition proceed to CLIP ranking for “at the beach.” A single ambiguous face cannot satisfy both people. Query resolution reuses stored vectors and does not rerun enrollment inference.

The threshold is an initial, uncalibrated operating point. Pose, lighting, face size, occlusion, and similar-looking people can cause errors.

11 OCR and Text Retrieval

11.1 Recognition and Caching

The application uses bundled ML Kit Latin Text Recognition. Its assets are packaged by the Android dependency, so OCR does not require a separate ONNX model file or an initial runtime download. The repository identifies the cache model as `mlkit-latin-16.0.1-vi-v1`.

PhotoTextRepository decodes an EXIF-oriented, full-frame image with a longest side of at most 2048 pixels. The larger OCR input preserves more small lettering than the semantic thumbnail. Recognized Unicode text and a search-normalized version are written to Room with the photo revision, model version, and truncation state.

Successful scans containing no text are cached. Failures remain eligible for a later pass, and changed photos or model versions invalidate old text. At most 100,000 characters are stored per image, with truncation surfaced in the reader. On-demand reading can recognize a photo that has not yet completed background OCR.

11.2 Word Matching

The index normalizes recognized text for word matching, including lowercasing text and turning punctuation into word separators. Original text remains available for display and copying. User input is converted to quoted literal terms for the FTS4 query; it is not treated as executable FTS syntax. A query permits at most 64 distinct terms.

Table 10. OCR Query Semantics and Boundaries

Example input	Search interpretation	Boundary
invoice total	Match the words “invoice” and “total.”	All terms must match.
70.000	Match normalized number tokens separated by punctuation.	Does not validate a monetary amount or parse bill fields.
@alex invoice	Resolve the saved identity and require the OCR terms.	Requires face indexing and an enrolled matching handle.
A translated synonym	Match only the entered words after normalization.	No translation, fuzzy spelling, or semantic OCR matching.

11.3 Search Behavior and User Value

OCR search does not invoke the CLIP text encoder, and OCR matches do not influence Normal-query ranking. OCR retrieval uses stored text independently of visual embeddings; background indexing processes OCR last. Candidate reads omit image-vector bytes when OCR does not need them. An empty text-match set returns early.

Date sorting is applied while selecting OCR matches, giving the true oldest/newest 100 across all matching pages. The reader displays the original recognized text and supports selection/copying. Sharp printed text and screenshots are suitable inputs; tiny text, blur, handwriting, and poor contrast can lower recognition quality.

12 Incremental Indexing and Reliability

12.1 Work Scheduling and Per-photo Progress

PhotoIndexScheduler enqueues unique WorkManager work with a battery-not-low constraint and the KEEP policy. Repeated requests do not create one worker per photo. The worker queries accessible MediaStore images newest-first and compares media IDs and modification times with persisted index state. Networking and charging are not required.

Table 11. Incremental Indexing Decisions

Photo condition	Worker behavior
New photo	Create the necessary metadata, run pending OCR, visual, and face stages, and persist successful results.
Unchanged photo	Reuse completed stages; backfill missing or outdated OCR and face indexes.
Modified photo	Invalidate older dependent data and compute results for the new revision.
Removed or inaccessible photo	Reconcile the accessible MediaStore snapshot and remove obsolete index rows. A fully denied permission state is detected before treating the library as empty.
Unreadable photo	Record/log the failure and continue processing other images. Refresh provides another attempt at pending work.

Within a photo, visual embedding and face work are attempted before OCR, which is indexed last. Each stage has its own cached completion information. This permits OCR and visual search to become useful incrementally rather than waiting for an entire library-wide OCR pass. A missing visual model can be reported while OCR processing continues.

12.2 Consistency During Asynchronous Work

Successful writes are committed as processing advances, so an interruption does not discard previously completed photos. At the end of a pass, the worker compares a fresh MediaStore snapshot with the starting snapshot. A changed library requests another incremental pass.

Room transactions compare parent photo revisions before attaching OCR or face results. A visual write older than the current parent revision is also rejected. Completing a visual embedding for the same revision retains text already scanned for that photo. These rules address consistency risks arising from concurrent library refresh, on-demand OCR, and background inference.

12.3 Resource Control and Cancellation

The worker uses a controlled sequence rather than launching a coroutine for every photo. Semantic thumbnails are 256×256 ; face decoding and OCR use separate bounded full-frame images. Bitmaps are released as processing progresses. CLIP inference sessions are retained and use two intra-op threads; model calls are serialized with mutexes.

Coroutine cancellation is checked between work units and during candidate scanning. ML Kit's task cannot itself be cancelled, so the recognizer waits for that task to finish before its bitmap can be recycled. WorkManager constraints can delay background processing; the UI therefore separates waiting from running and preserves useful saved progress.

13 Technology Stack and Configuration

The technology inventory below records the versions declared in the repository. The Kotlin Compose plugin version is distinguished from the Android Gradle Plugin's built-in Kotlin integration.

Table 12. Declared Technology Versions and Roles

Technology	Declared version	Purpose
Android Gradle Plugin	9.2.1	Android compilation and APK packaging.
Gradle wrapper	9.4.1	Reproducible build entry point.
Kotlin Compose plugin	2.2.10	Kotlin-based Compose integration.
KSP	2.2.10-2.0.2	Room source generation.
Compose BOM / Material 3	2025.08.01	Declarative UI and component styling.
Navigation / Lifecycle	2.9.3 / 2.9.3	Destinations, ViewModel, lifecycle-aware state.
Room / SQLite FTS4	2.7.2	Persistent index, migrations, literal text search.
WorkManager	2.10.3	Constrained, persistent background indexing.
Coil Compose	3.3.0	Loading device media images into Compose.
ONNX Runtime Android	1.22.0	Local CLIP and face-model inference.
ML Kit text recognition	16.0.1	Bundled offline Latin text recognition.
JUnit / coroutine tests	4.13.2 / 1.10.2	Unit assertions and asynchronous test control.
AndroidX JUnit / Espresso	1.3.0 / 3.7.0	Android instrumentation support.

13.1 SDK and JVM Settings

The application has `minSdk = 29`, `targetSdk = 36`, and a compile SDK of API 36 with minor API level 1. It uses application ID `com.example.mobile_image_retrieval`, versionCode 1, and versionName 1.0. Gradle daemon criteria select **JDK 21**, while Java source and target compatibility are **17**. The daemon runtime and Java compilation target serve distinct toolchain roles.

13.2 Bundled Model Assets

Table 13. Bundled ONNX Model Assets

Asset inspected	Size (MB)	Role
<code>mobileclip2_s0_image.onnx</code>	45.57	Image encoder
<code>mobileclip2_s0_text.onnx</code>	253.89	Text encoder
<code>det_500m.onnx</code>	2.52	Face detector
<code>w600k_mbf.onnx</code>	13.62	Face recognizer

Sizes use decimal MB and describe raw files, not installed memory use. The four graphs total approximately 315.60 MB, before tokenizer assets, OCR, libraries, and APK overhead. ONNX assets are packaged uncompressed. The generated model configuration also records its Python export environment; those exporter versions are separate from Android runtime dependencies.

14 Development Environment and Build Procedure

14.1 Prerequisites

Use Android Studio with support for the project’s Android Gradle Plugin, the Android SDK platform matching compile API 36.1, SDK Platform-Tools, and an Android 10+ device or emulator. Configure JDK 21 for Gradle’s daemon criteria. The project compiles Java-compatible sources for level 17. Use the included Gradle wrapper rather than substituting a different Gradle installation.

Initial SDK, Gradle, and dependency setup can require internet access on the development computer. This is separate from the application’s offline runtime behavior. Allow sufficient disk space for the model files, dependency caches, build outputs, and an emulator if used.

14.2 Prepare the Source and Assets

1. Obtain the submitted source tree or clone the project repository, then open its root directory in Android Studio. The root contains `settings.gradle.kts` and `gradlew`.
2. Use SDK Manager to install the matching Android SDK components. Let Android Studio write the machine-specific `local.properties`, or set its `sdk.dir` to the actual local SDK path. This file is intentionally gitignored.
3. Confirm the seven model/configuration files below are present in `app/src/main/assets/models/`. The two large CLIP ONNX graphs are gitignored and must accompany a source-only submission separately.

```
mobileclip2_s0_image.onnx
mobileclip2_s0_text.onnx
mobileclip2_s0_config.json
mobileclip2_s0_vocab.json
mobileclip2_s0_merges.txt
det_500m.onnx
w600k_mbf.onnx
```

Use the CLIP graphs, tokenizer files, and configuration from the same verified export. Do not mix model checkpoints. If regeneration is necessary, inspect `tools/export_mobileclip2.py` and regenerate the export as one set. OCR assets arrive through the bundled ML Kit dependency and are not an additional manual file download.

14.3 Build From the Project Root

On Linux or macOS, the wrapper can be invoked through Bash even when its executable bit is absent:

```
bash ./gradlew :app:assembleDebug
bash ./gradlew :app:testDebugUnitTest :app:lintDebug
```

On Windows, use `.\gradlew.bat` in place of `bash ./gradlew`. Android Studio can also sync and run the application. A successful debug build produces `app/build/outputs/apk/debug/app-debug.apk`. No backend server, API key, or remote database configuration is required.

15 Installation and Operating Procedures

15.1 Install and Open the Application

For an emulator, launch an Android 10+ virtual device. For a physical device, enable USB debugging and accept the computer's debugging authorization. From the project root:

```
adb devices
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell am start -n \
    com.example.mobile_image_retrieval/.MainActivity
```

Alternatively, copy the APK to the device and open it with Android's package installer, allowing installation from that source if prompted. Updating an existing installation requires the same application ID and a compatible signing key. The `-r` option preserves existing app data when a compatible update succeeds; uninstalling first removes the private index.

15.2 First-run Procedure

1. Grant photo access. Android 10–12L uses `READ_EXTERNAL_STORAGE`; Android 13+ uses `READ_MEDIA_IMAGES`. Android 14+ also supports selected-photo access through `READ_MEDIA_VISUAL_USER_SELECTED`.
2. Start with a small accessible set containing a clear scene, a single-face portrait, and a legible screenshot or receipt. Open Albums to confirm actual device photos are visible.
3. Observe OCR and visual indexing progress. A battery-not-low constraint can leave background work queued. Search completed photos while remaining stages continue.
4. Try Normal query with an English scene description, OCR search with known visible words, and a saved handle after enrolling a person. Then open a result and use Read text or Save a copy.
5. Close and reopen the same installation to check retained people, history, and indexing progress. Search and browse in airplane mode after assets are packaged and access is granted.

15.3 Common Problems and Recovery

Table 14. Common Operating Problems and Recovery Procedures

Symptom	Action
Model unavailable	Check both CLIP graphs and matching config/tokenizer files; rebuild the APK after correcting assets.
No photos or incomplete library	Check permissions, selected-photo scope, and Albums refresh. A fresh emulator may have no photos.
Few search results	Wait for the relevant indexing stage, then use Search again. Try words or scenes actually present in accessible photos.
Person query rejected	Add the handle in People or use Update face for an outdated reference. Choose a clear single-face enrollment photo.
Save a copy fails	Check readable source access and free storage, then retry. The implementation attempts to remove an incomplete destination.
Install reports a signature conflict	Obtain a build signed with the existing key to retain app data; do not assume an unrelated debug key can update it.

Saved copies are written to `Pictures/PhotoSearch` and shown in Recently Added by addition time. Deletion uses Android’s consent mechanism where required. Sharing opens compatible external apps only when the user requests it.

16 Testing Methodology and Validation Results

16.1 Recorded Execution Results

The execution results below are reported in the project audit dated 5 September 2026, as documented in `COURSE_REQUIREMENTS.md`. They are reproduced as recorded validation evidence; no independent Android test execution was conducted as part of this document review.

Table 15. Validation Results Reported in the Project Audit

Recorded check	Recorded outcome
Debug build	Passed; four ONNX assets bundled uncompressed. Universal debug APK: 502,532,800 bytes.
JVM unit tests	63 passed, 0 failures.
Android instrumentation	40 passed, 0 failures on Android 16 / API 36.1 emulator.
Android lint	0 errors; 21 dependency/API-version and KTX-style warnings.
Manual continuity check	Viewer retained the selected photo on rotation. After process termination/relaunch, the “sunset” draft/history returned with 475/475 photos still indexed.
Cold visual query	Approximately 8.2 seconds on the evaluated emulator, including model initialization. The loading UI remained available.

The recorded checks establish coverage of specified functional and reliability scenarios. The latency measurement represents a single cold query and does not characterize a distribution or sustained indexing throughput. Statistical retrieval accuracy and cross-device performance require separate evaluation.

16.2 Test Coverage

Unit tests cover vector math, embedding output validation, exact top-K ranking, deterministic ties, OCR date ordering across pages, mention parsing, distinct face assignment, text normalization, date bounds, and incremental planning. Instrumentation covers Room migrations/cascades, revision protection, OCR search on metadata-only photo rows, real bundled OCR/CLIP/face inference, copy success/failure cleanup, filter restoration, and landscape viewer actions.

16.3 Supplementary Source and Asset Verification

Source inspection counted **63 unit-test annotations across 13 files** and **40 instrumentation-test annotations across 13 files**, consistent with the audit’s counts. The two CLIP graphs, vocabulary, and merges file were present and their SHA-256 hashes matched the generated configuration. The supplied face inspection script read both ONNX contracts and hashes; those hashes matched FACE_MODELS.md.

The supplementary review verified source structure and asset consistency. Build, lint, and runtime outcomes remain attributed to the project audit; the asset checks do not constitute an additional inference benchmark.

16.4 Reproducing Runtime Checks

After completing Section 14, run the following tasks with the wrapper on a dedicated test emulator:

```
:app:testDebugUnitTest
:app:lintDebug
:app:connectedDebugAndroidTest
```

The project’s `android.injected.androidTest.leaveApksInstalledAfterRun=true` setting retains installed APKs after connected tests to reduce unintended loss of a development index.

17 Self-Assessment Against the Course Rubric

17.1 Assessment Method

The self-assessment maps implemented capabilities and documented evidence to the course marking rubric. A capability is classified as implemented when it is supported by an identifiable source component and relevant audit or test evidence.

Table 16. Self-Assessment Against the Course Marking Rubric

Criterion	Weight	Self-assessment and remaining evidence
Functional completeness	30%	Implemented: text/image/combined visual search, enrolled people, OCR, filters, albums, viewing, text reading, sharing, deletion, and copying. Audit records successful build and tests. Recognition remains approximate; unsupported videos and album editing are outside reported scope.
Technical quality and source	25%	Separated interface, ViewModel, and repository responsibilities; explicit Room migrations; coroutine-based execution; WorkManager scheduling; runtime permissions; and revision checks. Tests address failure and lifecycle scenarios. Physical-device and large-library profiling require further evaluation.
UI and UX	20%	Eight connected feature screens; two clear query modes; adaptive grids, light/dark themes, zoom, landscape actions, and distinct loading/empty/error/access states. No formal usability study is supplied.
Originality and complexity	15%	Integration of offline cross-modal retrieval, distinct-person matching, OCR/FTS, persistence, and Android media operations using pretrained models. Comparative originality and suitability for the confirmed group size remain subject to assessor review.
Report, presentation, collaboration	10%	Required report subjects, demonstration protocol, and work-division table are documented. Contributor attribution and evidence of video and presentation delivery remain outstanding.

17.2 Completed Application Work

The implementation meets the explicitly supplied capability categories: connected screens, local storage, permitted native stack, installable Android packaging, and device integration. The source and course audit identify no missing feature against those explicit implementation requirements. Successful persistence, storage operations, indexing, and real-model paths have specific recorded tests.

17.3 Overall Assessment

Version 1.0 provides an integrated Android implementation of offline visual, identity-based, and text-based photo retrieval. The documented results support completion of the specified application capabilities. Further work should prioritize representative device evaluation, retrieval-quality measurement, and completion of contributor and presentation records.

18 Limitations and Future Development

18.1 Current Practical Limits

Bundling models enables offline first use but produces a large installation artifact. The audit reports a universal debug APK of approximately 502.5 MB. First initialization and full-library indexing can be slow, particularly on an emulator.

Normal-query visual descriptions work best in English. OCR can miss small, blurred, handwritten, or poorly contrasted text. Face detection can miss small or obscured faces, and uncalibrated identity matching can produce false matches.

Table 17. Architectural Trade-Offs

Design choice	Benefit and cost
Exact local vector scan	Straightforward, deterministic candidate ranking with bounded vector pages; scan time grows with the number of candidates. The top-100 cap limits the displayed set.
Separate OCR and visual modes	Clear task semantics and independent progress; users must choose the mode that matches their intent.
Explicit person enrollment	User-selected names and references; no automatic discovery, clustering, or naming of everyone in the library.
Local private database	No application upload path and useful offline persistence; uninstall/clear-data removes the application’s index, people, and history.
Full-frame bounded OCR/face decode	Better retention of faces and text than a cropped semantic thumbnail; additional decoding cost and resolution-dependent misses.

The product indexes accessible images only. Video frame sampling, temporal video search, favorites, automatic scene splitting, and album editing are not implemented. The repository documents restrictions on the supplied face weights, so deployment beyond the documented research use requires resolving model rights separately.

18.2 Prioritized Future Work

1. **Measure target-device behavior.** Record cold/warm query latency, indexing photos per minute, peak memory, database growth, and battery use on representative phones and library sizes.
2. **Evaluate retrieval quality.** Build a consented, representative evaluation set with relevant-photo labels. Measure visual recall/precision, OCR term retrieval, and face false matches/misses before selecting thresholds.
3. **Reduce resource cost.** Evaluate a verified accelerated or smaller export, configurable indexing policy, ABI-specific packaging, and an explicit FP16 storage migration, preserving model-contract checks.
4. **Extend only from measured needs.** Add video indexing or album editing as separately scoped features. Consider an approximate local index only if profiling shows exact scanning is insufficient.